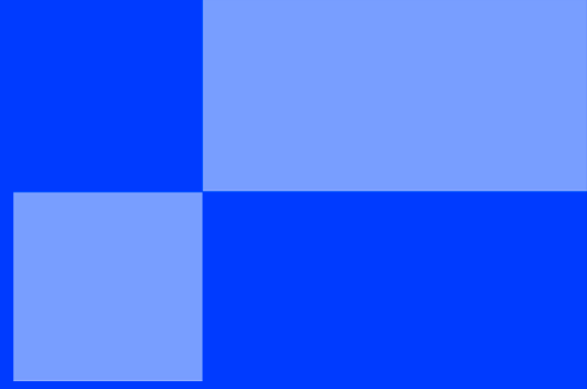
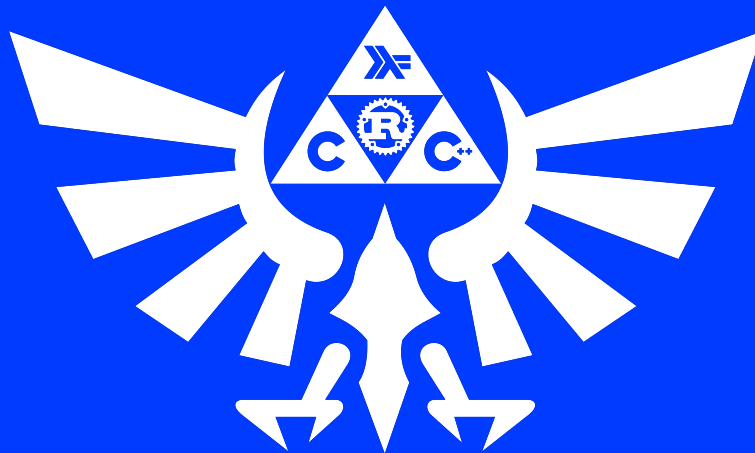




DAY 05



< I UNDERSTOOD THAT REFERENCE />



DAY 05

All your exercises will be compiled with `clang++` and the `-std=c++20 -Wall -Wextra -Werror` flags, unless specified otherwise.

All output goes to the standard output, and must be ended by a newline, unless specified otherwise.



None of your files must contain a `main` function, unless specified otherwise. We will use our own `main` functions to compile and test your code. It will include your header files.

For each exercise, **the files must be turned-in at the root of your repository unless specified otherwise.**



Read the examples CAREFULLY. They might require things that weren't mentioned in the subject...



The `*alloc`, `free`, `*printf`, `open` and `fopen` functions, as well as the `using namespace` keyword, are forbidden in C++. By the way, `friend` is forbidden too, as well as any library except the standard one.

Unit Tests

It is highly recommended to test your functions as you implement them. It is common practice to create and use what are called **unit tests**.

From now on, we expect you to write unit tests for your functions (when possible). To do so, please follow the instructions in the **"How to write Unit Tests"** document on the intranet, available [here](#).

For them to be executed and evaluated, put a [Makefile](#) at the root of your directory with the `tests_run` rule as mentioned in the documentation linked above.

Here is a sample set of unit tests for the **string** class :

```
#include <riterion/criterion.h>

Test(string, default_value)
{
    std::string s;
    cr_assert_eq(s, "");
}

Test(string, assign)
{
    std::string s;

    s = "test";
    cr_assert_eq(s, "test");
}

Test(string, append)
{
    std::string s("test");

    s += "ing";
    cr_assert_eq(s, "testing");
}
```

Exercise 0 - The Mondasians



Turn in : `Human.hpp`, `Human.cpp`

What are you doing here ? You shouldn't have come on Mondas ! The twin planet of Earth is dying, its inhabitant rely on technology to survive. For how long ?...

First, you need to create the Mondas inhabitants: Mondasians. Quite surprisingly, they are humans ! Except they are far more intelligent than us.

Create a `Human` class in the `Mondas` namespace. It contains the two following `private` member variable:

- ✓ `_name`: a `std::string` given at construction
- ✓ `_iq`: an `unsigned int` optionally given at construction with a default value of 192.

At construction, it prints:

`[NAME]: I must save Mondas with my IQ of [IQ].`

At destruction, it prints:

`[NAME]: I failed to save my world...`

You must provide getters for both member variables: `getName` & `getIq`.

Additionally, Mondasians have developped the technology to increase (and *only* increase) their IQ using the `setIq` method. It prints:

`[NAME]: My IQ is now [IQ].`

If the new IQ given as parameter is lower or equal than the current one, it does nothing.

Mondasians spend most of their time thinking on how to save their world. Add a `think` methods that prints the following:

`[NAME]: Think, think, think...`



Replace `[NAME]` and `[IQ]` with the name and IQ of the mondasian.

Here is a sample `main` function and its expected output :

```
int main(void)
{
    Mondas::Human human1("Korving", 164);
    const Mondas::Human human2("Joy");

    std::cout << human1.getName() << std::endl;
    std::cout << human2.getIq() << std::endl;
    human1.setIq(193);
    human1.setIq(103);
    std::cout << human1.getIq() << std::endl;
    human1.think();
    human2.think();

    return 0;
}
```

```
Terminal
$> clang++ -std=c++20 -Wall -Wextra -Werror *.cpp && ./a.out | cat -e
Korving: I must save Mondas with my IQ of 164.$
Joy: I must save Mondas with my IQ of 192.$
Korving$
192$
Korving: My IQ is now 193.$
193$
Korving: Think, think, think...$
Joy: Think, think, think...$
Joy: I failed to save my world...$
Korving: I failed to save my world...$
```



Methods should be `const` when they don't modify the object.

Exercise 1 - The Cybermen



Turn in : `Human.hpp`, `Human.cpp`, `Cyberman.hpp`, `Cyberman.cpp`

Mondasians can no longer survive on their home planet. They must adapt to overcome the now hostile environment... Mondasians must upgrade themselves to cybernetically augmented humanoids.

Create a `Mondas::Cyberman` class, a cybernetic armor for Mondas inhabitants. It contains a private `Mondas::Human& _human` private member variable initialized at construction.

At construction, it prints:

`[CNAME]: Unit activated.`

Upon destruction, it prints:

`[CNAME]: Unit deactivated.`

We must be able to access its internal human for maintenance purpose:

- ✓ `Mondas::Human& getHuman()`: get a reference of the human contained.
- ✓ `const Mondas::Human& getHuman()const`: same as non-const method.

Just like the human, the Cyberman can `think()`:

`[CNAME]: Pain, pain, pain...`

What? Pain?! That was not supposed to happen! We need a device to suppress feelings and emotions to make our cybermen perfect. Create a `Mondas::Cyberman::Inhibitor` class that contain a single private `bool` value to know if the device is active with a default value of `false`. It contains a `set(bool)` and `get()` method to change and check its value.

The `Inhibitor` is placed directly into the `Human`. Add an optional `Mondas::Cyberman::Inhibitor*` parameter to the `Mondas::Human` constructor.

Add theses methods to the human:

- ✓ `void setInhibitor(Mondas::Cyberman::Inhibitor*)`: set the inhibitor to the human, does nothing if the human already have one.
- ✓ `Mondas::Cyberman::Inhibitor* getInhibitor()`: get the human's inhibitor, or `nullptr` if none is set.
- ✓ `const Mondas::Cyberman::Inhibitor* getInhibitor()const`: same as non-const method.



You might run into an include problem called cyclic include. Ever heard of forward declaration ?

Let's get back to our `Mondas::Cyberman::think` method... Now, it should print the following if an inhibitor is set and active in its human:

`[CNAME]: Computation in progress.`

Otherwise:

`[CNAME]: Pain, pain, pain...`

We also need to know the IQ of the Cyberman :

`unsigned int getIq()const`

It returns the IQ of the contained human if its inhibitor is set and active, 0 otherwise.



Replace `[CNAME]` with the name of the human in the Cyberman, converted to l33t with the following rules :

- ✓ 'a' and 'A' are replaced by '4'
- ✓ 'b' and 'B' are replaced by '8'
- ✓ 'e' and 'E' are replaced by '3'
- ✓ 'g' and 'G' are replaced by '6'
- ✓ 'i', 'I', 'l' and 'L' are replaced by '1'
- ✓ 'o' and 'O' are replaced by '0'
- ✓ 'r' and 'R' are replaced by '2'
- ✓ 's' and 'S' are replaced by '5'
- ✓ 't', 'T', 'y' and 'Y' are replaced by '7'
- ✓ ' ' (spaces) are replaced by '_'

Here is a sample `main` function and its expected output :

```
int main(void)
{
    Mondas::Cyberman::Inhibitor inhibitor;
    Mondas::Human human1("Korving", 164, &inhibitor);
    Mondas::Human human2("Voga");

    {
        Mondas::Human human3("Lumic", &inhibitor);
    }

    const Mondas::Cyberman unit1(human1);
    unit1.think();
    inhibitor.set(true);
    std::cout << unit1.getIq() << std::endl;
    unit1.think();

    Mondas::Cyberman unit2(human2);
    unit2.think();
    unit2.getHuman().setInhibitor(&inhibitor);
    unit2.think();

    return 0;
}
```



```
Terminal
$> clang++ -std=c++20 -Wall -Wextra -Werror *.cpp && ./a.out | cat -e
Korving: I must save Mondas with my IQ of 164.$
Voga: I must save Mondas with my IQ of 192.$
Lumic: I must save Mondas with my IQ of 192.$
Lumic: I failed to save my world...$
K02v1n6: Unit activated.$
K02v1n6: Pain, pain, pain...$
164$
K02v1n6: Computation in progress.$
V064: Unit activated.$
V064: Pain, pain, pain...$
V064: Computation in progress.$
V064: Unit deactivated.$
K02v1n6: Unit deactivated.$
Voga: I failed to save my world...$
Korving: I failed to save my world...$
```


Exercise 2 - The Cyberleader



Turn in : [Human.hpp/cpp](#), [Cyberman.hpp/cpp](#), [Cyberleader.cpp/hpp](#)

It's too late for Mondas ! Once a Cyberleader is created, they will use every resources available to upgrade everything. This world is doomed...

Just like the Cyberman, the Cyberleader contains a human given at construction.

The Cyberleader was designed with simplicity and minimalism in mind : its used only through [upgrade](#) methods. You must deduce it's internal functioning with the following example :

```
int main(void)
{
    Mondas::Cyberman::Inhibitor inhibitor;
    Mondas::Human human("Lumic", 201);
    Mondas::Cyberleader cyberleader(human);

    Mondas::Cyberman::Inhibitor testInhibitor;
    Mondas::Human testHuman("Voga");
    Mondas::Cyberman testCyberman(testHuman);

    cyberleader.upgrade(testCyberman);
    cyberleader.upgrade(testHuman);
    cyberleader.upgrade(testInhibitor);
    human.setInhibitor(&inhibitor);
    cyberleader.upgrade(testCyberman);
    inhibitor.set(true);

    std::cout << "---" << std::endl;

    cyberleader.upgrade(testHuman);
    cyberleader.upgrade(testCyberman);
    cyberleader.upgrade(testInhibitor);

    std::cout << "---" << std::endl;

    testCyberman.getHuman().setInhibitor(&testInhibitor);
    cyberleader.upgrade(testHuman);
    cyberleader.upgrade(testCyberman);
    cyberleader.upgrade(testInhibitor);

    return 0;
}
```

```
Terminal
$> clang++ -std=c++20 -Wall -Wextra -Werror *.cpp && ./a.out | cat -e
Lumic: I must save Mondas with my IQ of 201.$
1um1c: Master unit activated.$
Voga: I must save Mondas with my IQ of 192.$
V064: Unit activated.$
1um1c: Pain, pain, pain...$
1um1c: Pain, pain, pain...$
1um1c: Pain, pain, pain...$
1um1c: Pain, pain, pain...$
---$
1um1c: Upgrading human 'Voga'.$
Voga: My IQ is now 384.$
IQ of unit doubled, increased to 384.$
No inhibitor installed.$
1um1c: Upgrading unit 'V064'.$
1um1c: Upgrading human 'Voga'.$
Voga: My IQ is now 768.$
IQ of unit doubled, increased to 768.$
No inhibitor installed.$
1um1c: Activating inhibitor.$
---$
1um1c: Upgrading human 'Voga'.$
Voga: My IQ is now 1536.$
IQ of unit doubled, increased to 1536.$
1um1c: Inhibitor already activated.$
1um1c: Upgrading unit 'V064'.$
1um1c: Upgrading human 'Voga'.$
Voga: My IQ is now 3072.$
IQ of unit doubled, increased to 3072.$
1um1c: Inhibitor already activated.$
1um1c: Inhibitor already activated.$
V064: Unit deactivated.$
Voga: I failed to save my world...$
1um1c: Master unit deactivated.$
Lumic: I failed to save my world...$
```

Exercise 3 - The Cyberfoundry



Turn in : `Human.hpp/cpp`, `Cyberman.hpp/cpp`, `Cyberleader.hpp/cpp`, `Cyberfoundry.hpp/cpp`

They began the mass production of Cybermen. Every living human is taken to the Cyberfoundries by force to be converted. They call it an upgrade, I see the end of humanity on Mondas.

The Cyberfoundries are meant to convert humans into Cybermen and Cyberleaders. They need a copious amounts of energy to run. First, the `Cyberfoundry` need to transform natural resources into raw energy.

The natural resources of Mondas are represented by a simple `unsigned int` passed as parameter to the `Cyberfoundry` constructor. Be careful, multiple `Cyberfoundry` will use the resources of Mondas :

- ✓ `Mondas::Cyberman* buildCyberman(Mondas::Human&):` cost (50 + human's IQ) energy to build a `new` Cyberman from the given human.
- ✓ `Mondas::Cyberleader* buildCyberleader(Mondas::Human&):` cost (150 + 2 x human's IQ) energy to build a `new` Cyberleader from the given human.

Both methods return `nullptr` if not enough energy is available. The expected output can be seen in the example below. A `new` inhibitor must be installed on each unit if the human doesn't already have one. Inhibitors are always enabled when a unit leave the Cyberfoundry.



If the `Human` already have an `Inhibitor`, it was created using `new`.

Cyber units can be recycled to regain energy. The unit and its inhibitor are `deleted` to recycle their components :

- ✓ `void recycle(Mondas::Cyberman*&):` regain 50 energy.
- ✓ `void recycle(Mondas::Cyberleader*&):` regain 150 energy.

The unit's pointer is then set to `nullptr`.

Here is an example of how to use a Cyberfoundry:

```
int main(void)
{
    unsigned int nature = 777;
    Mondas::Human human1("Korving", 164);
    Mondas::Human human2("Voga", 89);
    Mondas::Human human3("Lumic", 201);
    Mondas::Cyberfoundry cyberfoundry(nature);

    auto cyberman = cyberfoundry.buildCyberman(human1);
    auto cyberleader = cyberfoundry.buildCyberleader(human3);

    cyberfoundry.buildCyberman(human2);
    cyberfoundry.buildCyberleader(human2);
    cyberfoundry.recycle(cyberman);
    cyberfoundry.recycle(cyberleader);
    cyberfoundry.recycle(cyberleader);
    cyberman = cyberfoundry.buildCyberman(human2);
    cyberfoundry.recycle(cyberman);

    return 0;
}
```

```
Terminal
$> clang++ -std=c++20 -Wall -Wextra -Werror *.cpp && ./a.out | cat -e
Korving: I must save Mondas with my IQ of 164.$
Voga: I must save Mondas with my IQ of 89.$
Lumic: I must save Mondas with my IQ of 201.$
A new Cyberfoundry has opened.$
Building a new Cyberman from 'Korving'.$
K02v1n6: Unit activated.$
Building a new Cyberleader from 'Lumic'.$
1um1c: Master unit activated.$
Not enough energy to build a new Cyberman.$
Not enough energy to build a new Cyberleader.$
Recycling Cyberman 'K02v1n6'.$
K02v1n6: Unit deactivated.$
Recycling Cyberleader '1um1c'.$
1um1c: Master unit deactivated.$
Nothing to recycle.$
Building a new Cyberman from 'Voga'.$
V064: Unit activated.$
Recycling Cyberman 'V064'.$
V064: Unit deactivated.$
A Cyberfoundry has closed.$
Lumic: I failed to save my world...$
Voga: I failed to save my world...$
Korving: I failed to save my world...$
```

Exercise 4 - The Cyberlegion



Turn in : `Human.hpp/cpp`, `Cyberman.hpp/cpp`, `Cyberleader.hpp/cpp`, `Cyberfoundry.hpp/cpp`, `Cyberlegion.hpp/cpp`

The Cyberlegion is the ultimate evolution of the Cybermen ! Mondas is now fully enslaved, the production factory are running all day... Soon, they will send colony ship to conquer Earth.

The Cyberlegion is a group of Cybermen lead by a Cyberleader. It can be of any size : a single Cyberman, or billions. It has its own Cyberfoundry to produce even more units. Each Cyberlegion is unique and has its own identification code. The first ever Cyberlegion has the ID #1, the second one the ID #2, and so on...

Create the `Mondas::Cyberlegion` class, taking the number of natural resources as parameter at construction. At construction, it prints:

```
Cyberlegion #[ID]: Humanity will be upgraded.
```

At destruction, it prints:

```
Cyberlegion #[ID]: Impossible, we can't fail.
```

Then delete the entire legion.

`Humans` can be assimilated by the `Cyberlegion` :

```
bool assimilate(Mondas::Human* human)
```

The first `Human` assimilated will become the `Cyberleader` of the `Cyberlegion`, all the other ones will be converted to simple `Cybermen`. After creating the `Cyberleader`, it prints:

```
Cyberlegion #[ID]: '[CNAME]'is now our Cyberleader.
```

After creating a `Cyberman`, it is upgraded using the `Cyberleader`, and then prints:

```
Cyberlegion #[ID]: '[CNAME]'has joined our Cyberlegion.
```

They are created using the `Cyberfoundry`. If it fails, it prints:

```
Cyberlegion #[ID]: '[NAME]'can't be assimilated. '[NAME]'will be deleted.
```

Then deletes the useless `Human`.

The method returns `true` if the `Human` was assimilated, `false` otherwise.



`Cybermen` must be stored in an array of `Cyberman*`, allocated using the `new[]` operator.

Add the following getters :

- ✓ `Mondas::Cyberleader* getCyberleader();`
- ✓ `const Mondas::Cyberleader* getCyberleader() const;`
- ✓ `Mondas::Cyberman** getCybermen();`
- ✓ `const Mondas::Cyberman* const * getCybermen() const;`

✓ `unsigned int getSize() const;`

The size of the Cyberlegion is the number of `Cyberman` it contains.

Finally, we need to know the computational power of the `Cyberlegion`. It's the sum of the IQ of its Cybermen.

✓ `unsigned int getIq()const;`

Here is a sample `main` function and its expected output :

```
int main(void)
{
    unsigned int energy = 1337;

    {
        Mondas::Cyberlegion cyberlegion(energy);
    }

    Mondas::Cyberlegion cyberlegion(energy);

    std::srand(42);
    while (cyberlegion.assimilate(new Mondas::Human("Citizen " + std::to_string(std::rand
        () % 100), 128 + std::rand() % 128)) == true);
    std::cout << cyberlegion.getSize() << std::endl;
    return 0;
}
```

```
Terminal
$> clang++ -std=c++20 -Wall -Wextra -Werror *.cpp && ./a.out | cat -e
A new Cyberfoundry has opened.$
Cyberlegion #1: Humanity will be upgraded.$
Cyberlegion #1: Impossible, we can't fail.$
A Cyberfoundry has closed.$
A new Cyberfoundry has opened.$
Cyberlegion #2: Humanity will be upgraded.$
Citizen 66: I must save Mondas with my IQ of 228.$
Building a new Cyberleader from 'Citizen 66'.$
C171z3n_66: Master unit activated.$
Cyberlegion #2: 'C171z3n_66' is now our Cyberleader.$
Citizen 81: I must save Mondas with my IQ of 169.$
Building a new Cyberman from 'Citizen 81'.$
C171z3n_81: Unit activated.$
C171z3n_66: Upgrading unit 'C171z3n_81'.$
C171z3n_66: Upgrading human 'Citizen 81'.$
Citizen 81: My IQ is now 338.$
IQ of unit doubled, increased to 338.$
C171z3n_66: Inhibitor already activated.$
Cyberlegion #2: 'C171z3n_81' has joined our Cyberlegion.$
Citizen 12: I must save Mondas with my IQ of 134.$
```

```
Terminal
Building a new Cyberman from 'Citizen 12'.$
C171z3n_12: Unit activated.$
C171z3n_66: Upgrading unit 'C171z3n_12'.$
C171z3n_66: Upgrading human 'Citizen 12'.$
Citizen 12: My IQ is now 268.$
IQ of unit doubled, increased to 268.$
C171z3n_66: Inhibitor already activated.$
Cyberlegion #2: 'C171z3n_12' has joined our Cyberlegion.$
Citizen 21: I must save Mondas with my IQ of 156.$
Building a new Cyberman from 'Citizen 21'.$
C171z3n_21: Unit activated.$
C171z3n_66: Upgrading unit 'C171z3n_21'.$
C171z3n_66: Upgrading human 'Citizen 21'.$
Citizen 21: My IQ is now 312.$
IQ of unit doubled, increased to 312.$
C171z3n_66: Inhibitor already activated.$
Cyberlegion #2: 'C171z3n_21' has joined our Cyberlegion.$
Citizen 35: I must save Mondas with my IQ of 159.$
Not enough energy to build a new Cyberman.$
Cyberlegion #2: 'Citizen 35' can't be assimilated. 'Citizen 35' will be deleted.$
Citizen 35: I failed to save my world...$
3$
Cyberlegion #2: Impossible, we can't fail.$
C171z3n_66: Master unit deactivated.$
Citizen 66: I failed to save my world...$
C171z3n_81: Unit deactivated.$
Citizen 81: I failed to save my world...$
C171z3n_12: Unit deactivated.$
Citizen 12: I failed to save my world...$
C171z3n_21: Unit deactivated.$
Citizen 21: I failed to save my world...$
A Cyberfoundry has closed.$
```

v 3.1

{EPITECH}